

Programming Language Principles, Design, and Implementation

Compilers - Code Generation

Vincent Rahli

University of Birmingham

Where are we?

- ▶ Part 1: Principles of programming
- ▶ **Part 2: Compilers**

Today

Code Generation

- ▶ Assembly
- ▶ Basic Blocks
- ▶ Flow Graphs
- ▶ Code Generation

Further reading:

- ▶ Chapter 8 of the Dragon book
- ▶ Chapter 8–11 of “Modern Compiler Implementation”

Problem

Generate a (semantically equivalent) target program from the intermediate representation (IR).

The problem of generating an **optimal** target program from a given source program is **undecidable**!

In practice: compiler builders rely on **heuristics**.

To generate efficient target code, compilers rely on **optimizations**.

A code generator performs:

- ▶ **instruction selection**
- ▶ **register allocation**
- ▶ **instruction ordering**

It generates target code for a given architecture such as

- ▶ RISC (Reduced Instruction Set Computer): many registers, three-address instructions, simple addressing modes, simple instruction-set architecture
- ▶ CISC (Complex Instruction Set Computer): few registers, two-address instructions, variety of addressing modes, several register classes, variable-length instructions

Instruction Selection

Challenges

Instruction Selection: translating a high-level IR is straightforward. Producing efficient code is hard. For example:

$$x = y + z \mapsto \begin{array}{l} \text{LD } R0, y \\ \text{ADD } R0, R0, z \\ \text{ST } x, R0 \end{array}$$

Therefore:

$$\begin{array}{l} a = b + c \\ d = a + e \end{array} \mapsto \begin{array}{l} \text{LD } R0, b \\ \text{ADD } R0, R0, c \\ \text{ST } a, R0 \\ \text{LD } R0, a \\ \text{ADD } R0, R0, e \\ \text{ST } d, R0 \end{array}$$

The second **load** is useless

Register assignment: NP-complete

Instruction ordering: Selecting the best order in which computations are performed is NP-complete

Target Language

We will consider

- ▶ a **simple assembly language**
- ▶ for a three-address machine (opcode + 3 addresses: 1 destination and 2 sources)
- ▶ byte-addressable machine
- ▶ n general-purpose registers

Instructions:

- ▶ **LD** $dst, addr$: loads the value at the address $addr$ into the location dst
- ▶ **ST** dst, r : stores the value in register r into the location dst
- ▶ **op** dst, src_1, src_2 , where $op \in \{\text{ADD}, \text{SUB}, \dots\}$: applies the operation op to the values in locations src_1 and src_2 and places the result in the location dst
- ▶ **BR** L : branches to the machine instruction with label L
- ▶ **Bcond** r, L , where $cond \in \{\text{LTZ}, \dots\}$: branches to L if the value in register r satisfies the condition $cond$

Target Language

Addressing modes:

- ▶ a **location** can be:
 - ▶ a variable name x , referring to the memory location reserved for x
 - ▶ an indexed address of the form $a(r)$, where a is a variable, and r a register: adds the value in register r to a (useful to access arrays: a is the base address, and r contains an offset).
 - ▶ an integer indexed by a register of the form $n(r)$: adds the value in register r to n (useful for pointer arithmetic)
- ▶ indirect addressing modes:
 - ▶ $*r$: the memory location found in the location represented by the content of register r
 - ▶ $*n(r)$: the memory location found in the location represented by the content of register r to which we add n
- ▶ immediate constant addressing mode of the form $\#n$

Target Language

Arithmetic: The TAC $x = y - z$ can be translated into:

```
LD R1, y
LD R2, z
SUB R1, R1, R2
ST x, R1
```

We might be able to omit some instructions, e.g., if y is already in a register.

Jumps: the statement `if  $x < y$  goto  $L$` can be translated into:

```
LD R1, x
LD R2, y
SUB R1, R1, R2
BLTZ R1, L
```

where L is the label that represents the 1st machine instruction generated from the TAC instruction labeled L .

Target Language

Pointers: The statements $x = *p$ and $*p = y$ can be translated into, respectively:

LD $R1, p$	LD $R1, p$
LD $R2, 0(R1)$	LD $R2, y$
ST $x, R2$	ST $0(R1), R2$

Arrays: Let a be an array of 8-byte elements. The TACs $b = a[i]$ and $a[j] = c$ can be translated into, respectively:

LD $R1, i$	LD $R1, c$
MUL $R1, R1, 8$	LD $R2, j$
LD $R2, a(R1)$	MUL $R2, R2, 8$
ST $b, R2$	ST $a(R2), R1$

Target Language

Generate target code for

$$\begin{aligned}x &= a * b \\ y &= x + c\end{aligned}$$

The target code is:

```
LD R1, a
LD R2, b
MUL R1, R1, R2
ST x, R1
LD R3, x
LD R4, c
ADD R3, R3, R4
ST y, R3
```

Instruction Costs

Problem: the problem of generating an **optimal** target program from a given source program is **undecidable**.

Solution: use heuristics to minimize the **cost** of generated instructions.

A simple **cost** function:

- ▶ all instructions have a default cost of 1
- ▶ accessing a register has a cost of 0
- ▶ accessing a memory location or a constant has a cost of 1

Examples:

- ▶ LD $R0, R1$ has a cost of 1
- ▶ LD $R0, M$, which loads the content of the memory location M , has a cost of 2

Instruction Costs

What are the costs of LD $R1$, *100($R2$) and of the translation of the TAC $x = y + z$?

- ▶ Cost of LD $R1$, *100($R2$): 3
- ▶ Translation of $x = y + z$:

LD $R1$, y

LD $R2$, z

ADD $R1$, $R1$, $R2$

ST x , $R1$

Cost: 7

Procedures

Consider a procedure q that calls a procedure p as follows:

```
param  $x_1$   
...  
param  $x_n$   
call  $p, n$ 
```

When q calls p

- ▶ control needs to be given to p
- ▶ once p is done, control needs to return to q

p returns by calling `return  $y$` or `return`

Procedures

When q calls p , it stores its return address in the generated activation record for p . Assume for simplicity that it is the first component of this activation record. The target code can be:

```
ST  $p.staticArea$ ,  $\#next$   
BR  $p.codeArea$ 
```

where

- ▶ $p.staticArea$ is the address of the beginning of p 's activation record, where the return address is stored
- ▶ $\#next$ is the return address, i.e., the address of the instruction following the above branch instruction
- ▶ $p.codeArea$ refers to the address of p 's first instruction in the **Code** area

When p returns, control is given back to q by jumping to the return address, i.e.,:

```
BR  $*p.staticArea$ 
```

Procedures

p.staticArea can be implemented by a register *SP* that stores the address of the current stack frame.

Initialization:

ST *SP*, *#stackStart*

where *#stackStart* is the address of the beginning of the stack.

If we do this, a **procedure call** would be:

ADD *SP*, *SP*, *#caller.recordSize*

ST **SP*, *#next*

BR *callee.codeArea*

SUB *SP*, *SP*, *#caller.recordSize*

where *caller.recordSize* is the size of an activation record, and *next* is the address of the instruction following the above “branch” instruction.

Return:

BR **0(SP)*

Procedures

Example:

```
    ...  
100 : ...           // q's code is stored at address 100  
120 : ST 364, #140   // save return address  
132 : BR 200         // call p  
140 : ...  
  
    ...  
200 : ...           // p's code is stored at address 200  
220 : BR *364        // return to address saved in location 364  
  
    ...  
300 :               // beginning of activation record for q  
  
    ...  
364 : 140           // beginning of activation record for p  
    ...
```

Basic Blocks

Basic blocks are used to help with **instruction selection**. They form the nodes of **flow graphs** which are used to generate more optimized intermediate code.

A **basic block** is a sequence of three-address instructions such that:

- ▶ There are no jumps into the middle of the block
- ▶ no instruction in the block halts or branches, except the last one

Partitioning algorithm: Given a sequence of three-address instructions:

- ▶ Identify the **leaders** (first instructions of blocks):
 - ▶ the first instruction in the sequence
 - ▶ any instruction that is the target of a (conditional) jump
 - ▶ any instruction that immediately follows a (conditional) jump
- ▶ A basic block consists of a leader, plus all the following instructions up to (excluding) the next leader (or the end of the sequence)

Basic Blocks

Partition the following sequence into basic blocks:

```
(1)   $i = 1$   
(2)   $j = 1$   
(3)   $t_1 = 10 * i$   
(4)   $t_2 = t_1 + j$   
(5)   $t_3 = 8 * t_2$   
(6)   $t_4 = t_3 - 88$   
(7)   $a[t_4] = 0.0$   
(8)   $j = j + 1$   
(9)  if  $j \leq 10$  goto (3)  
(10)  $i = i + 1$   
(11) if  $i \leq 10$  goto (2)  
(12)  $i = 1$   
(13)  $t_5 = i - 1$   
(14)  $t_6 = 88 * t_5$   
(15)  $a[t_6] = 1.0$   
(16)  $i = i + 1$   
(17) if  $i \leq 10$  goto (13)
```

```
(1)   $i = 1$   
(2)   $j = 1$   
(3)   $t_1 = 10 * i$   
(4)   $t_2 = t_1 + j$   
(5)   $t_3 = 8 * t_2$   
(6)   $t_4 = t_3 - 88$   
(7)   $a[t_4] = 0.0$   
(8)   $j = j + 1$   
(9)  if  $j \leq 10$  goto (3)  
(10)  $i = i + 1$   
(11) if  $i \leq 10$  goto (2)  
(12)  $i = 1$   
(13)  $t_5 = i - 1$   
(14)  $t_6 = 88 * t_5$   
(15)  $a[t_6] = 1.0$   
(16)  $i = i + 1$   
(17) if  $i \leq 10$  goto (13)
```

Flow Graphs

A **flow graph** has

- ▶ basic blocks as nodes
- ▶ edges that capture the control flow between blocks

There is an **edge** from block B_1 to B_2 if and only if

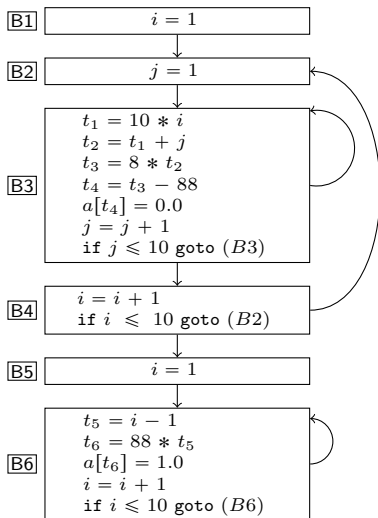
- ▶ there is a (conditional) jump from the end of B_1 to the beginning of B_2
- ▶ or B_2 immediately follows B_1 in the original sequence, and B_1 does not end in an unconditional jump

We also replace jumps to instruction numbers by jumps to blocks.

Flow Graphs

Example:

- (1) $\underline{i = 1}$
- (2) $\underline{j = 1}$
- (3) $\underline{t_1 = 10 * i}$
- (4) $\underline{t_2 = t_1 + j}$
- (5) $\underline{t_3 = 8 * t_2}$
- (6) $\underline{t_4 = t_3 - 88}$
- (7) $\underline{a[t_4] = 0.0}$
- (8) $\underline{j = j + 1}$
- (9) **if** $j \leq 10$ **goto** (3)
- (10) $\underline{i = i + 1}$
- (11) **if** $i \leq 10$ **goto** (2)
- (12) $\underline{i = 1}$
- (13) $\underline{t_5 = i - 1}$
- (14) $\underline{t_6 = 88 * t_5}$
- (15) $\underline{a[t_6] = 1.0}$
- (16) $\underline{i = i + 1}$
- (17) **if** $i \leq 10$ **goto** (13)



Code Generation

We will see how to generate code for basic blocks.

The generator tries to avoid generating unnecessary loads and stores through an efficient use of registers.

Uses of registers:

- ▶ some instructions require some or all the operands to be registers
- ▶ temporaries
- ▶ sharing between blocks
- ▶ for run-time storage management (e.g., *SP*)

The generation algorithm maintains the following **key datastructures**:

- ▶ **a register descriptor**: keeps track of the variables that currently have their values in registers
- ▶ **an address descriptor**: keeps track of the locations where the current values of variables are stored

Code Generation

We assume a `getReg( $I$ )` function, which given an instruction I , returns registers for each memory locations associated with I . We will see later how to implement it.

Algorithm for operations: given an instruction I of the form $x = y + z$ (and similarly for the other operations):

- ▶ use `getReg( $I$ )` to select registers R_x, R_y, R_z for x, y, z
- ▶ if y is not in R_y (according to the register descriptor), then generate `LD  $R_y, y'$` , where y' is one of the memory locations for y (according to the address descriptor)
- ▶ do the same for z
- ▶ generate `ADD  $R_x, R_y, R_z$`

Algorithm for copy: given an instruction I of the form $x = y$:

- ▶ we assume that `getReg( $I$ )` returns a single register R
- ▶ if y is not already in R (according to the register descriptor), generate `LD  $R, y$`
- ▶ if y is in R , do nothing

Code Generation

With the above algorithm, it can happen that a variable's value is only stored in a register

- ▶ do nothing if it is a temporary only used in the block
- ▶ otherwise, it needs to be stored: for each variable x such that the address descriptor does not say that its value is stored in x 's memory location, generate `ST  $x$ ,  $R$` , where R is a register that stores x 's value

Code Generation

The **register and address descriptors** can be updated as follows:

1. for the instruction **LD** R, x
 - ▶ **register descriptor**: R holds x only
 - ▶ **address descriptor**: R is a location for x
2. for the instruction **ST** x, R
 - ▶ **address descriptor**: x is a location for itself
3. for the instruction **ADD** R_x, R_y, R_z
 - ▶ **register descriptor**: R_x holds x only, and only R_x holds x
 - ▶ **address descriptor**: x 's only location is R_x (maybe not in x itself)
 - ▶ **address descriptor**: remove R_x from the records for all variables other than x
4. **special case**: when processing a copy statement of the form $x = y$, in addition to rule 1
 - ▶ **register descriptor**: R now holds x , and only R holds x
 - ▶ **address descriptor**: x 's only location is R (maybe not in x itself)

Code Generation

How does this generator translates this block to target code?

$$\begin{aligned}
 t &= a - b \\
 u &= a - c \\
 v &= t + u \\
 a &= d \\
 d &= v + u
 \end{aligned}$$

It is translated as follows:

statement	instructions	register descriptors			address descriptors						
		R1	R2	R3	a	b	c	d	t	u	v
					a	b	c	d			
$t = a - b$	LD R1, a LD R2, b SUB R2, R1, R2	a	t		a, R1	b	c	d	R2		
$u = a - c$	LD R3, c SUB R1, R1, R3	u	t	c	a	b	c, R3	d	R2	R1	
$v = t + u$	ADD R3, R2, R1	u	t	v	a	b	c	d	R2	R1	R3
$a = d$	LD R2, d	u	a, d	v	R2	b	c	d, R2		R1	R3
$d = v + u$	ADD R1, R3, R1	d	a	v	R2	b	c	R1			R3
end	ST a, R2 ST d, R1	d	a	v	a, R2	b	c	d, R1			R3

Selecting Registers

The above algorithm relies on `getReg( $I$ )` to select registers given I . If I is $x = y + z$ for example, how do we select a register R_y for y ?

1. if y is already in a register, return one of those
2. otherwise, return an empty register
3. otherwise, if there is no empty register, reuse one: R . Let v be a variable in R according to the register descriptor. We need to make sure that v 's value is still accessible before reusing R (for all vs in R) – and remove R from v 's entry in the address descriptor:
 - ▶ stop if the address descriptor says that v is somewhere else besides R
 - ▶ if v is x and x is not z – we need a register for z too – then stop (x 's old value is not going to be used)
 - ▶ if v is not going to be used later, then stop
 - ▶ otherwise, generate `ST  $v$ ,  $R$` to store v in itself (i.e., perform a **spill**)

How do we select a register R_x for x ? This is done as for y , except:

1. only a register that holds x only is acceptable in step 1. above, since a new value of x is being computed
2. if y is not used after I , and R_y contains only y after being loaded, selecting R_y is also acceptable

Conclusion

What did we cover today?

- ▶ Code Generation
- ▶ Assembly
- ▶ Basic Blocks
- ▶ Flow Graphs
- ▶ Code Generation

Next time?

- ▶ Code Optimization